Pit Lane Manager — Production Blueprint for an Unlicensed F1 Management Sim

The opportunity is real and the window is narrow. Motorsport Manager 4 sits at 4.0★ with 18 months of stagnation, F1 Manager (Frontier) was cancelled after a £15M write-down, and F1 Clash monetizes aggressively with loot boxes that drew an ASA ruling — meanwhile the F1 audience has exploded to 826M fans (+90M YoY), skews younger and more female post-*Drive to Survive*, and the 2026 season launches in late February. A lean fork of Dynasty Manager, executed in 8 weeks, can land a differentiated F2P career sim with no loot boxes, a livery editor, and short 3–5 minute races into a market that has exactly one live-ops winner and one dormant incumbent. The rest of this document is the full build plan: MM4 system teardown, legal-safe naming, Dynasty→F1 migration, a complete GDD, a Claude Code prompt library, a week-by-week roadmap, and the ASO/launch strategy.

---

1. Motorsport Manager 4 — deep feature teardown

MM4 (Playsport/Miniclip, Sep 2023, **$4.99 premium + optional IAP**, iOS 4.0★/238, Play 4.1★/~1.2k) is essentially **MM3.5** — new 3D cars, a seasonal R&D tech tree, a Pit Strategy planner, a new Race Strategist staff role, sprint-race weekends, and a redesigned 3D HQ, TapTap but the same core engine and player complaints. The series' signature moat is **three disciplines (Open Wheel, GT, Endurance) across 3–4 tiers with 9 total championships**, a political GMA voting layer, and a paid Game Editor that functions as official modding.

Race weekend and live race

Practice sessions build per-compound "Setup Knowledge" and driver feedback; qualifying uses a **pole-shootout elimination** format (reverse-grid in some lower series); optional **sprint races** added in MM4; race length is Short/Medium/Long and functions as a de-facto difficulty slider. During the race players toggle **Driving Style** (Attack 1.4× tire wear → Back-Up 0.6×), Steam Community **Engine Mode** Steam Community (Super-Overtake pink → Low blue, with fuel and heat side-effects), Motorsport Manager PC Wiki and plan a **Pit Strategy sequence** of stop laps, compounds, and fuel loads that can be overridden live. PlayMods.net Tire compounds span **Ultra-Soft → Hard** plus Intermediate/Wet, each with wear/temperature/grip state and a cliff below ~25% grip. Motorsport Manager PC Wiki Steam Community **ERS** has supplier choice (battery/supercapacitor/flywheel) Motorsport Manager PC Wiki and deployment modes; weather is dynamic with per-lap probability transitions and accuracy gated by a Forecast Centre building.

Car development

Parts populate **7 slots** (Engine, Gearbox, Chassis, Front Wing, Rear Wing, Brakes, Suspension; GT drops front wing), each with Performance and Reliability scores. The **seasonal R&D tech tree** (new in MM4) replaces MMM3's random research pop-ups with a branching unlock graph. **Pre-season Chassis Design** picks suppliers that trade off tire wear vs fuel economy vs improvability.

End-of-season part values **carry over** as next season's baseline (Motorsport Manager PC Wiki) — a core long-run progression mechanic. (Steam Community)

Drivers, staff, team

Nine 1–20 driver stats: Braking, Cornering, **Smoothness** (tire wear), Overtaking, **Consistency** (lap-time variance), Adaptability (setup/wet), Fitness, Feedback, Focus (Motorsport Manager PC Wiki) — plus Morale, Marketability (gates sponsor tiers), hidden Improvability, and a large Traits library. Staff are Chief Designer, per-driver Race Engineer, Mechanics (6 sub-stats), Head of Scouting, and the **new Race Strategist**. (Google Play) HQ has **15 buildings** (Factory, Design Centre, Wind Tunnel, Simulator, Forecast Centre, Helipad — required for 5★ sponsors (Steam Community) — Visitor Centre, Theme Park, Road Car Factory, etc.) capping at a 4.5★ rating. (Steam Community)

Finances

Income comes from **6 sponsor slots** (3 fixed + 3 race-bonus), tiered 1★–5★ and gated by average marketability; seasonal **prize money pools** ($500M WMC, $250M APSC, $125M ERS); (Steam Community) Chairman's Payment (risk-adjusted); and HQ buildings. Expenses are staff salaries, part builds (exponential cost curve), chassis design allocation, and travel.

What players love and what they complain about

Praise: clean UI, depth-without-overwhelm, the R&D/strategist additions, no aggressive monetization (148Apps: *"IAPs are invisible to me"*), (148apps) Game Editor longevity, the three-discipline scope. **Complaints, in frequency order**: "carbon copy of MM3" (top Google review with 114 helpful votes), severe bugs (frozen race screens, (apple) flickering background, (google) (Google Play) AI qualifying double-stacking, constructor-name leaderboard bug carried since MM3), (apple) (App Store) **weak AI** (doesn't recognize rain in qualifying, pits last-lap), (apple) a **forced tier regression** that punishes staying at one level for >2 seasons, (apple) (App Store) missing MM3 features, **no custom race lengths**, (google) no interval timing in race view, no track preview, cannot view opponent stats, (apple) a feature freeze since April 2024, and a team-name character cap **reduced from 30 to 14** (App Store) — a small regression that became a lightning-rod.

Differentiators vs F1 games

MM4's unique hooks are **multi-discipline scope, GMA political voting on rules, supplier-based chassis design, and the paid Game Editor**. F1 Clash is shallower but licensed and live-service; F1 Manager (Frontier) is cancelled; iGP Manager is online-only. (Igpmanager) (GTPlanet) The **#1 highest-leverage improvement** a competitor can ship is **stronger race AI** — it's the most consistent complaint across every MM review generation.

---

2. Unlicensed naming — the legal-safe playbook

The Motorsport Manager approach is the empirical gold standard: a decade of shipping with an "expy" grid that evokes real F1 teams (TV Tropes) without trademark overlap, and zero known lawsuits from FOM, FIA, or any manufacturer. The rules are simple: **never use "F1," "Formula 1,"**

"Formula One," "FIA," "Grand Prix" as brand elements; change team names, logos, three-letter codes, and at least one dominant livery color from any real team; avoid Davids-v.-Riot-style identity stacking (matching name + nationality + stats + look = infringement even with a different name); keep real country/city names (public domain) but rename the circuits themselves (Silverstone®, Circuit de Monaco, GTPlanet Spa-Francorchamps are all registered marks). Ship a **"not affiliated with Formula 1, the FIA, or any real team or driver"** disclaimer on splash/store/credits to defeat Lanham §43(a) false-endorsement theories.

Ten teams for the starting grid

#	Team	Inspired by	HQ	Primary/Secondary	Principal	Tier
1	**Scuderia Cavallini**	Ferrari	Modena, IT	#8B0E14 / #F0E9D4	Matteo Brambilla	Top
2	**Nordstern Grand Prix**	Mercedes AMG	Stuttgart, DE	#4A4E52 / #00A0A0	Dr. Henrik Waldmann	Top
3	**Taurex Racing**	Red Bull	Graz, AT	#0B1A36 / #E8A317	Max Reinthaler	Top
4	**Papaya Works**	McLaren	Guildford, UK	#F26A1B / #5DB8E8	Nigel Ashcroft	Mid→Top
5	**Equipe Azurene**	Alpine	Chartres, FR	#1E4AA8 / #EE2B7C	Céline Vardanyan	Mid
6	**Harwick Racing**	Aston Martin/Williams	Silverstone area, UK	#1F4F2E / #A3E635	Lionel Harwick III	Mid
7	**Sakura Dynamics**	Honda/RB	Sakura, JP	#F5F2EB / #C8102E	Akiko Tanabe	Mid
8	**Corsa Montello**	Sauber/Stake	Hinwil, CH	#7ED321 / #111111	Andrea Brülhart	Back→Mid
9	**Atlas Grand Prix**	Haas	Kannapolis, US	#6B7280 / #D0342C	Chuck Ballard Jr.	Back
10	**Kestrel Motorsport**	Racing Bulls	Faenza, IT/UK	#102A43 / #F9E71E	Piero Calandri	Back

Twenty drivers

Cavallini: Lorenzo Fazzini (IT, 27, volatile quali-pace) / Miguel Otero (ES, 31, methodical veteran). **Nordstern**: Nile Harker (UK, 34, seven-time winner showman) / Elias Brenner (IT, 19, rookie phenom). **Taurex**: Maxim Vandersteen (NL, 27, race animal) / Yuki Ohashi (JP, 25, fan

favorite). **Papaya**: Jonah Pemberton (UK, 25, clutch in wet) / Oscar Lindqvist (AU/SE, 23, cold tire manager). **Azurene**: Esteban Roquet (FR, 28, angular racer) / Pierre Carbonell (FR, 29, fan darling). **Harwick**: Felix Kingsley (UK, 40, graying sage, 2× champ) / Lance Ainsworth (CA, 26, wet-weather pay driver). **Sakura**: Kaito Morimura (JP, 22, metronome) / Issak Haddad (FR/DZ, 20, fiery junior champ). **Corsa Montello**: Nico Ferraz (BR, 23, overtaking flair) / Gabriel Strand (SE, 36, reliable points). **Atlas GP**: Ollie Crenshaw (UK, 20, combative rookie) / Esteban Ruiz (MX, 35, street-circuit specialist). **Kestrel**: Isak Hallgren (NO, 21, quiet) / Daniyar Alimov (KZ, 24, one-lap speed pay driver).

Ten circuits for the calendar (renamed venues at real locations)

Bahrain Desert Park (Sakhir, March, hot/dry) → Jeddah Marina Boulevard (SA, March, night street) → Albert Harbour Circuit (Melbourne, April, showers) → Riviera Street Circuit (Monaco, May, 30% rain risk) → Catalan Ring (Montmeló, June, hot/high-deg) → Austria Alpen Ring (Spielberg, July, mountain storms) → Northampton National (Silverstone area, July, frequent rain) → Ardennes Forest Circuit (Stavelot, August, microclimate) → Autodromo di Brianza (Monza area, September, dry) → Interlagos Heights (São Paulo, November, tropical rain). For MVP ship these 10; expand to 18 in v1.1.

Livery rules and typography

Pull colors from **national flags + one off-real accent** rather than team Pantones. Avoid any real-team logo motifs (bulls, prancing horses, stars-in-rings, Mercedes' three-pointer, McLaren's speedmark swoosh). Use three-letter codes that don't collide with **FER, MER, MCL, RBR, ALP, AST, WIL, ALF, HAA, SAU**. Fonts: **Orbitron, Rajdhani, Saira, Exo 2, Titillium Web, Bank Gothic** — all free, all motorsport-coded, all trademark-clean. Never use Formula1 Display, Ferrari Sans, or the Mercedes AMG corporate face.

---

3. Dynasty Manager → F1 migration plan

Dynasty Manager (React 18 / Vite / TS / Tailwind / Zustand / Framer Motion / shadcn-ui / Capacitor) maps onto F1 management with high fidelity for ~70% of its systems. The remaining 30% — tactics, match engine, injuries — gets torn out and replaced with a race engine and car-development module.

Mapping table

Dynasty Manager	Pit Lane Manager	Action
Club/Team	Team/Constructor	**Rename**
Squad/Roster	Grid (2 drivers + reserve)	**Rename + reduce size**
Player	Driver	**Rename**

Dynasty Manager	Pit Lane Manager	Action
Player contract	Driver contract	**Keep, adjust salary model**
Transfer market	Driver market	**Rename + add pay-drivers**
Season/Fixtures	Season/Race calendar	**Keep**
Match	Race weekend	**Rewrite** (tactics→strategy, match engine→race engine)
Training	Driver development + simulator	**Adapt**
Scouts	Scouts	**Keep**
Finances	Finances	**Keep + add per-race sponsor bonuses**
Sponsors	Sponsors	**Extend** (title + major + minor, race-bonus slots)
Board expectations	Board expectations	**Keep**
League table	Drivers' + Constructors' championships	**Extend**
Stadium	Factory/HQ	**Rename + redesign upgrade tree**
Coaching staff	Engineers (TD, race engineer, strategist, chief designer, mechanics)	**Adapt roles**
Kit/shirt sponsors	Car liveries + title sponsors	**Extend with livery editor**
Tactics screen	Race strategy (pit plan, tires, fuel, engine modes)	**Rewrite**
Match engine	Race engine (per-lap tick sim)	**Rewrite**
Injuries	Part reliability / DNFs	**Rework concept**
Youth academy	F2/F3 reserve drivers	**Adapt**
Cup competitions	Sprint weekends	**Rework**
N/A	**Car development / parts / upgrades / R&D tree**	**New module**

File-structure audit (what to keep, modify, delete, rewrite)

Keep as-is: `components/ui/*` (all shadcn primitives), `lib/utils.ts`, `hooks/useHydration.ts`, `lib/storage.ts` (Capacitor Preferences adapter), `routes.tsx` scaffolding, Tailwind/Vite/Capacitor config. **Modify/rename**: `components/game/team/*` (roster→grid, player-card→driver-card), `components/game/calendar/*`, `components/game/finance/*`, `stores/slices/teamSlice.ts`, `stores/slices/financeSlice.ts`, `stores/slices/careerSlice.ts`, `data/*.json` (replace player pools with driver pools, add circuits). **Delete**: tactics module, match engine, formations, injuries, set pieces, anything football-specific in `components/game/match/*` and `sim/match.ts`. **Rewrite from scratch**: `sim/race.ts`, `sim/lapTime.ts`, `sim/tires.ts`, `sim/overtake.ts`, `sim/pitStrategy.ts`, `sim/weather.ts`, `sim/reliability.ts`, `components/game/race/*`, new `components/game/cardev/*` module, new `stores/slices/raceSlice.ts`, new `stores/slices/carDevSlice.ts`.

State management migration

Consolidate into **one bound Zustand store** combining slices (official recommendation for cross-slice atomicity — signing a driver must debit finance AND add contract AND update roster in a single `set()`). Keep `persist + immer + devtools` middleware at the combined level, never inside slices. Zustand Use **branded IDs** (`type DriverId = string & { __brand: 'DriverId' }`) for type safety. Flatten entity storage as `Record<Id, Entity>` not nested trees. Bump `version` and write a `migrate()` function for every schema change post-launch. **Hydration gate is mandatory** — Zustand persist is async, and rendering the game before `useGameStore.persist.hasHydrated()` returns true will cause blank-screen crashes on iOS review and Guideline 2.1 rejections.

Data model (abbreviated — full version in the Claude prompt library below)

Core entities: **Team** (id, name, tier, driverIds[], carId, staffIds[], sponsorIds[], facilities, finance, prestige, boardExpectation), **Driver** (stats: pace/overtaking/defence/consistency/smoothness/feedback/wetSkill/potential, xp, morale, marketability, contractId, career record), **Contract** (driverId, teamId, seasons, salary, podiumBonus, championshipBonus, buyoutClause, signingBonus), **Car** (fittedParts by slot, inventory, overallPerformance, reliability, fuelCapacity, defaultSetup), **Part** (slot, performance, condition, maxLaps, lapsUsed, buildProgress), **Upgrade** (slot, projectedGain, progress, weeklyCost, risk, resultPartId), **Race** (round, circuitId, sessions, weatherBySession, result), **RaceResult** (rngSeed for deterministic replay, DriverRaceResult[] with optional lapTrace), **Championship** (driverStandings, constructorStandings), **SaveGame** (version, slotId, rngSeed, playerTeamId, currentDate, nextInterrupt, normalized entity tables, events, difficulty). Money is stored as **integer cents**; stats are branded 0–100 numbers.

---

4. Game Design Document

Core loop

Day-based calendar advance via a **Continue button** that processes scheduled events until it hits an interrupt (race weekend, contract offer, board meeting, upgrade complete). Race weekends drop into a sub-state machine: **Practice (tune setup, build compound knowledge) → Qualifying (pole shootout) → Race (live strategy)**. Post-race: results, points, sponsor payouts, driver morale updates, board reaction, news feed. Between races: spend on R&D (branching tech tree), build parts (factory queue), negotiate sponsors, scout drivers, train staff, upgrade HQ. Between seasons: driver market "silly season," sponsor renegotiation, chassis design for next year, tier promotion/relegation.

MVP v1 feature list (ship in 8 weeks)

One tier (Tier 1 only — 10 teams, 20 drivers, 10 circuits), career mode only, 2-driver teams, 10-race season, per-lap tick race sim with pit strategy planner and live overrides (driving style + tire compound + pit call), **5 driver stats** (pace, consistency, tire management, wet weather, overtaking — cut defence/feedback/potential to Stat v2), **5 part slots** (engine, aero, chassis, brakes, tires-compound-strategy), basic upgrade tree (linear per slot, not branching — branch in v1.1), 4 sponsor tiers, 4 HQ facilities (factory, wind tunnel, simulator, scouting center), basic driver market, contract negotiation (salary + length only), 3 difficulty levels, 3 race lengths (5/10/25 laps — the short default addresses MM4's biggest complaint), deterministic RNG replay, 1 save slot, livery color picker (primary + secondary only, no pattern editor), English only for first submission with Portuguese/Spanish/German/Italian/French prepped for week 10 update.

v1.1–v2 features (post-launch roadmap)

Tiers 2 + 3 (F2/F3 analogs with promotion/relegation) • full 9-stat driver system • branching R&D tech tree • livery editor with pattern layers • create-a-driver with helmet/suit • weekly live-ops event tied to real GP weekend (track + weather match) • historical fantasy seasons (1988/2005/2021) • async co-op leagues (4–12 friends) • 3 more HQ buildings • sprint weekends • reserve-driver system • 8 more circuits (18 total) • replay/share clips • Manager Pro subscription launches • additional languages.

Race weekend flow (MVP)

Practice (one condensed session, 2 minutes real-time): player picks a compound to test and a setup direction; game returns feedback points applied to a per-compound "Knowledge" meter (0–100%) that reduces tire-wear variance in the race. **Qualifying (single session, 90 seconds simulated)**: player picks one tire compound and one fuel load; game simulates one flying lap; top 10 set grid in order. **Race**: live view with track map, running order, gap column, lap counter, weather widget, player-driver HUD. Player can at any tick: change driving style (5 modes), change engine mode (3 modes for MVP — Conserve/Standard/Push), trigger pit (consumes next planned stop or free-calls a new one), change pit-stop plan for laps ahead. Pause/slow/fast time controls.

Driver stats (MVP 5-stat system)

Pace (0–100) — raw lap-time contribution, −0.03s per point vs a 50-reference driver. **Consistency** — inversely proportional to lap-time stddev; σ = 0.4s at 100, 1.1s at 0. **Tire Management** (smoothness) — reduces compound wear rate by up to 35% at 100. **Wet Weather** — multiplier in rain conditions, ±0.8s per lap swing between 0 and 100. **Overtaking** — adds to sigmoid in overtake-attempt resolution (also used as defense for simplicity in MVP). Each stat has XP 0–1000 that gains 1 level per 100 XP up to a hidden **potential** ceiling.

Car development tree (MVP linear, v1.1 branching)

Engine (4 tiers: Standard → Evolved → Competitor → Works) — contributes to straight-line pace. **Aero** (4 tiers) — contributes to cornering pace and weather sensitivity. **Chassis** (4 tiers) — contributes to overall balance and reliability. **Brakes** (4 tiers) — late-braking zone gain. **Tire Compound Strategy Research** (unlock access to Ultra-Soft and to Intermediate forecasting — this is meta, not a fitted part). Each upgrade costs money + weeks of development time; staff skill accelerates; **risk factor** means projected gain has ±20% variance. End-of-season carryover: best part becomes next season's baseline (copies MM4's mechanic).

Staff (MVP 3 roles)

Race Engineer (per driver) — in-race strategy bonus and practice feedback rate. **Chief Designer** — R&D weekly progress speed. **Sporting Director** — scouting quality + sponsor negotiation bonus. Each hireable from a pool, each 0–100 skill, each a salary cost. No mechanics/strategist split until v1.1.

Sponsor system and IAP integration

Four tiers: **Minor** (1★, available to everyone, ~$500K/season + $50K/race), **Major** (2★, unlocks at 25% marketability, ~$2M/season + per-podium bonus), **Premier** (3★, 60% marketability, ~$5M/season + per-win bonus), **Title** (4★, 85% marketability AND 4-star HQ, ~$15M/season + championship bonus). Six slots: **1 title + 2 premier + 3 minor/major**. IAP hook: a **Starter Sponsor Pack** ($2.99) gives a one-time 2-season Premier contract at marketability 40%, letting new players compete before earning it.

IAP design

- **Remove Ads** $3.99 one-time (kills the rewarded-ad-only complaint loop)
- **Starter Pack** $2.99 one-time (credit bundle + exclusive chrome livery + sponsor pack — drives first-purchase conversion)
- **Credit bundles** at the standard iOS price ladder: $0.99 / $2.99 / $4.99 / $9.99 / $19.99 / $49.99 / $99.99
- **Season Pass** $5.99 per 6-week season (free track + premium track with liveries, helmets, suits, driver portraits — priced below F1 Clash's $9.99 to feel fair)
- **Livery/Helmet/Suit packs** $2.99 themed

- **Manager Pro subscription** $4.99/mo or $34.99/yr (ad-free, +10% race rewards, daily currency stipend, exclusive avatar, 3 save slots)
- **Founder's Pack** $9.99 limited to first 90 days (1 month Pro + credit bundle + Founder livery + Founder badge)

Free-to-play economy that isn't predatory

Two currencies only — **Cash** (earned every race weekend, used for everything) and **Credits** (hard currency, earned slowly from achievements + purchased with IAP, used to rush upgrades or buy cosmetics). **No energy system, no wait-timer forcing currency spend, no loot boxes, no gacha.** Rush-upgrade pricing caps so a player can fully catch up within 2 weeks of ad-free play. Publicly commit **"No loot boxes. No pay-to-win. Ever."** on the store page and marketing — directly exploits F1 Clash's ASA loot-box ruling as differentiation.

---

5. Claude Code prompt library

Use these prompts verbatim with Claude Code in the order listed, one session per prompt. Each assumes the prior prompts' output is committed to the repo. Replace `{{PROJECT_ROOT}}` with your repo path.

Prompt 1 — Fork setup and cleanup

```
You are working in the Dynasty Manager codebase (React 18 / Vite / TS / Tailwind v4 / shadcn-ui / Zustand / Framer Motion / Capacitor 7). We are forking it into "Pit Lane Manager," an unlicensed F1 management sim.

Tasks:
1. Update package.json name to "pit-lane-manager", version 0.1.0, and description.
2. Update capacitor.config.ts: appId "com.pitlanemanager.app", appName "Pit Lane Manager".
3. Update index.html title and meta description.
4. Delete every file/folder specific to football: components/game/match/, components/game/tactics/, components/game/formations/, sim/match.ts, sim/tactics.ts, stores/slices/matchSlice.ts, stores/slices/tacticsSlice.ts, data/players.json (but KEEP data/teams.json as a template — we'll replace contents next prompt).
5. Rename everywhere in code (case-preserving): "club" → "team", "player" (when referring to athlete, not user) → "driver", "squad"/"roster" → "grid", "match" → "race", "fixture" → "race", "training" → "development", "manager" is fine to keep.
6. Update components/game/team/ filenames: PlayerCard → DriverCard, SquadGrid → GridView.
7. Remove all football-specific Tailwind classes, icons (soccer ball, goal, etc.), and copy.
8. Keep ALL shadcn primitives in components/ui/ untouched.
```

```
9. Keep stores/slices/{teamSlice, financeSlice, careerSlice, settingsSlice}.ts —
we'll modify their fields in the next prompt, not delete them.
10. Run `npm run build` and fix any resulting TS errors with minimal changes.
11. Commit with message: "chore: fork Dynasty Manager as Pit Lane Manager, remove
football-specific modules".

Output a summary of every file deleted, renamed, and modified at the end.
```

Prompt 2 — Core TypeScript domain models

```
Create src/types/game.ts with the complete F1 management sim domain model. Use
branded IDs for every entity ID. Use Money as integer cents (branded number), Stat
as branded 0-100 number. Include:

Enums/unions: TireCompound (ultraSoft|soft|medium|hard|intermediate|wet), PartSlot
(engine|aero|chassis|brakes|gearbox|suspension|frontWing|rearWing), StaffRole
(raceEngineer|chiefDesigner|sportingDirector|strategist|mechanic|scout),
SeriesTier (tier1|tier2|tier3), Weather
(dry|cloudy|lightRain|heavyRain|thunderstorm), SessionType
(practice|qualifying|race|sprint), DNFReason.

Entities (all as exported interfaces with JSDoc on every field explaining gameplay
meaning and value ranges): Driver (with nested DriverStats: pace, overtaking,
defence, consistency, smoothness, feedback, wetSkill, potential — all Stat 0-100),
Contract, Car (with CarSetup sub-interface), Part, Upgrade, Staff, Sponsor (with
bonus structure and requirements), Team (with TeamFinance and
FinancialTransaction), Circuit, Season (with regulations sub-object including
pointsTable, allowedCompounds, costCap, engineAllocation), Race (with
weatherBySession keyed map), DriverLapData, DriverRaceResult, RaceResult (with
rngSeed for deterministic replay), ChampionshipStanding, Championship,
ScheduledInterrupt, GameEvent, SaveGame (root — include version field for
migration, slotId, playerTeamId, currentDate, rngSeed, normalized Record<Id,
Entity> tables for every entity type, events array, difficulty config).

Rules:
- No nested entity references — always by ID.
- Every Money field is cents (integer).
- Every Stat field is branded 0-100.
- Add makeStat(n: number): Stat and makeMoney(cents: number): Money helper
constructors at the bottom.
- Do NOT include any functions or methods — pure types only.
- Keep the file under 600 lines. Include a header comment explaining the branded-
ID pattern and the normalized-store pattern.

After writing the file, run `npx tsc --noEmit` and verify zero errors.
```

Prompt 3 — Zustand store architecture

```
Build the Zustand store architecture in src/stores/. Use the slice pattern with a
single combined store, persist + immer + devtools middleware, and a Capacitor
Preferences storage adapter.

Create:
1. src/lib/storage.ts — export capacitorStorage: StateStorage using
@capacitor/preferences. Async getItem/setItem/removeItem. Handle missing keys
returning null.
2. src/stores/slices/teamSlice.ts — state: teams Record, actions: addTeam,
updateTeam, setPlayerTeam, adjustPrestige. Uses immer.
3. src/stores/slices/driverSlice.ts — state: drivers Record, contracts Record.
Actions: addDriver, signDriver (cross-slice: updates team roster + finance +
creates contract atomically via get()), releaseDriver, trainDriver (adds XP),
retireDriver.
4. src/stores/slices/raceSlice.ts — state: currentRaceId, currentSession,
liveRaceState (tick, lap, driverPositions, weather). Actions: startRaceWeekend,
advancePractice, runQualifying, tickRace, setDrivingStyle, setEngineMode,
callPitStop, finishRace.
5. src/stores/slices/careerSlice.ts — state: currentDate, currentSeasonId,
nextInterrupt, seasons Record, races Record, championships Record, events[].
Actions: advanceDay, advanceUntilInterrupt, completeSeason, startNewSeason.
6. src/stores/slices/financeSlice.ts — player team balance accessor + transaction
log. Actions: credit, debit, addTransaction. Cross-slice reads OK.
7. src/stores/slices/carDevSlice.ts — parts Record, upgrades Record. Actions:
startUpgrade, advanceUpgradesWeekly, completeUpgrade (produces Part), fitPart.
8. src/stores/slices/settingsSlice.ts — audio, locale, difficulty, hapticsEnabled.
9. src/stores/useGameStore.ts — combines all slices. Applies middleware IN THIS
ORDER (outermost first): devtools → persist → immer. Persist config: name "pit-
lane-save-v1", version 1, storage capacitorStorage via createJSONStorage,
partialize to strip ephemeral liveRaceState, onRehydrateStorage sets a hasHydrated
flag, migrate stub returning state as-is for v1.
10. src/hooks/useHydration.ts — returns boolean from
useGameStore.persist.hasHydrated.

Type each slice creator as StateCreator<GameState, [["zustand/devtools", never],
["zustand/persist", unknown], ["zustand/immer", never]], [], SliceState>. Export a
single useGameStore hook.

Rules:
- Never mutate outside immer set().
- signDriver must be atomic — if finance debit fails (insufficient funds), no
contract is created, no roster changes.
- All cross-slice reads use get(), not direct imports.
- Add a useShallow selector helper in src/stores/selectors.ts for common derived
```

```
data (standings, playerTeam, playerDrivers).

Run `npx tsc --noEmit` and `npm run build`. Zero errors expected.
```

Prompt 4 — Race simulation engine

```
Build the core race simulation engine in src/sim/. This is a pure-functional
module — no React, no Zustand, no side effects. Input: RaceState. Output: updated
RaceState + events[].

Files:
1. src/sim/rng.ts — seedable PRNG. Export mulberry32(seed: number): () => number
and splitmix32(seed: number): (salt: string) => number for derived per-lap seeds.
Include gaussianNoise(rng, sigma): number using Box-Muller.

2. src/sim/lapTime.ts — export computeLapTime(driver, car, setup, tire, fuelKg,
weather, circuit, rng): number (seconds). Formula:
   lapTime = circuit.baseLapTimeSec
           + carPerformanceFactor(car.overallPerformance)    // -0.05s per pt
above 50
           + driverPaceFactor(driver.stats.pace)             // -0.03s per pt
above 50
           + tirePhaseTime(tire.compound, tire.lapsUsed)     // see tires.ts
           + fuelMassPenalty(fuelKg)                         // 0.035s per kg
           + weatherPenalty(weather, tire.compound)          // e.g. slicks in
rain = +8s
           + setupQualityFactor(setup.qualityAtCurrentCircuit) // -0.5s at 100
quality
           + gaussianNoise(rng, driverConsistencySigma(driver.stats.consistency))
   Apply tireManagement from driver.stats.smoothness as a wear-rate multiplier
(handled in tires.ts call).

3. src/sim/tires.ts — 4-phase model (warmup, peak, degradation, cliff). export
tirePhaseTime(compound, lapsUsed): number and advanceTireWear(tire, drivingStyle,
smoothness, trackFactor): updatedTire. Softer compounds: shorter phases, faster
peak, steeper cliff. Compound constants:
   ultraSoft: peak=+0.8s, warmup=1 lap, peakEnd=6, degEnd=12, cliff=14+
   soft: peak=+0.4s, peakEnd=10, degEnd=20, cliff=24+
   medium: peak=0s, peakEnd=16, degEnd=28, cliff=34+
   hard: peak=-0.2s slower baseline but most durable, peakEnd=22, degEnd=40,
cliff=48+
   intermediate: for lightRain
   wet: for heavyRain

4. src/sim/overtake.ts — export attemptOvertake(attacker, defender, track, rng): {
success: boolean, timeCost: number }. Use:
```

   paceDelta = attackerCurrentPace - defenderCurrentPace
   skillDelta = attacker.overtaking - defender.defence (in MVP defence = overtaking for simplicity)
   p = sigmoid(paceDelta * 2 + skillDelta * 0.1 - track.overtakingDifficulty * 2)
   if rng() < p: swap positions, attacker loses 0.3s. Otherwise both lose 0.1s.
   Only attempt if gap < 1.0s.

5. src/sim/pitStrategy.ts — export shouldPit(driver, currentLap, state): PitDecision with reason. Rules:
   - If tireWear > 85% and not on final stint: yes ("tireWear")
   - If currentLap == plannedPitLap: yes ("planned")
   - If weather transition detected AND wrong tire type: yes ("weatherCall")
   - If safetyCarDeployed AND haven't used the pit stop under SC yet: yes ("safetyCar")
   - If undercut window open on driverAhead: yes ("undercut")
   - Else: no

6. src/sim/weather.ts — Markov chain. states: [dry, cloudy, lightRain, heavyRain]. Transition matrix with 0.92 stay probability, adjacent transitions 0.04 each. export advanceWeather(current, circuitVolatility, rng): Weather. export forecast(current, lapsAhead, circuit, rng): Weather[] (3-lap preview for player UI).

7. src/sim/reliability.ts — export rollFailure(car, drivingStyle, engineMode, ambientTempC, rng): { failed: boolean, reason?: DNFReason }. pFailure = basePartFailureRate * stressMultiplier(drivingStyle, engineMode) * tempMultiplier(ambientTempC). Engine failures dominate at push+hot temps.

8. src/sim/race.ts — export simulateRace(initialState, options): { result: RaceResult, lapTrace: DriverLapData[][] }. Per-lap tick loop over all drivers:
   - compute lap time
   - apply tire wear
   - check pit decision, if pit, add pit lane time loss
   - check reliability failure
   - after all drivers computed, resort grid by totalTime
   - resolve overtakes (for adjacent pairs where gap < 1.0s)
   - maybe advance weather
   - emit events (overtake, pit, dnf, fastestLap)
   Return final standings, points allocated from season.regulations.pointsTable, pole/fastestLap, headline string generated from top 3 + any DNF drama.

Rules:
- Every function is pure; RNG is passed in.
- All tunable constants live at the top of each file as exported SIM_CONSTANTS.
- Include unit-test hooks: each function exported independently.
- Write 5 Vitest smoke tests in src/sim/__tests__/race.test.ts verifying determinism (same seed = same result) and sanity (best car wins ~60%+ of simulated

```
identical-driver races over 1000 iterations).

Run `npm run test` and confirm green.
```

Prompt 5 — Live race UI

```
Build the live race screen in src/components/game/race/. Use shadcn-ui
Card/Button/Dialog/Select/Sheet primitives, Framer Motion LazyMotion with
domAnimation, and Tailwind.

Components:
1. RaceScreen.tsx — top-level layout. Grid: header (lap counter, weather, time
control), main (track map + standings), footer (player-driver HUD). useGameStore
selector for liveRaceState.

2. TrackMap.tsx — SVG top-down loop track (use a pre-baked SVG path per circuit in
data/circuits/). Render one dot per driver positioned along the path by
lapProgress %. Animate ONLY at lap boundaries using Framer Motion's layoutId,
never per tick. For MVP, a simplified oval or figure-8 SVG per circuit is
acceptable.

3. StandingsTable.tsx — shadcn Table component. Columns: Position, Driver (team-
color dot + name), Gap to leader (+X.Xs), Tire icon + laps on, Last lap time.
Highlight player's drivers. Virtualize with @tanstack/react-virtual if >20 rows
(won't be needed for tier1 MVP).

4. PlayerHUD.tsx — sticky bottom panel with two driver cards (for player's two
drivers). Each card: driver portrait placeholder, current position, tire + wear
bar, fuel bar, current driving style button, current engine mode button. Tapping
opens a Sheet with strategy controls.

5. StrategyPanel.tsx — shadcn Sheet. Sections:
   - Driving Style (radio: Conserve / Standard / Push / Attack — 4 options for
MVP)
   - Engine Mode (radio: Cool / Standard / Push — 3 for MVP)
   - Pit Plan (list of scheduled stops; tap to edit lap + compound; add/remove
stops)
   - Pit Now button (confirms: pits at next opportunity)
   - Dispatch via useGameStore actions (setDrivingStyle, setEngineMode,
updatePitPlan, callPitNow).
   Haptic feedback on every action (@capacitor/haptics Impact.Light).

6. TimeControls.tsx — header buttons: Pause / 1x / 2x / 5x. Maps to setInterval
tick rate. Pause sets interval = null.

7. RaceEvents.tsx — right-side ticker of recent events (overtakes, pits, DNFs,
```

```
fastest laps). Fade in/out with Framer Motion, max 5 visible. Pulled from
liveRaceState.recentEvents.

8. useRaceLoop.ts hook — owns the setInterval that calls
useGameStore.getState().tickRace() at the current speed. NEVER bind the tick
directly to Framer Motion — drive the sim in JS, push display updates only at lap
boundaries.

Rules:
- Wrap the whole race screen in <LazyMotion features={domAnimation} strict>.
- Animate only transform + opacity. Never width/height/top/left.
- React.memo every component that re-renders frequently.
- Use useShallow selectors to read liveRaceState.
- All long-lived DOM nodes have style={{ willChange: 'transform' }} only during
active animation.
- Respect useReducedMotion().
- Test on iPhone SE 2 and a budget Android to validate FPS.

Include an empty state for when liveRaceState is null (pre-race).
```

Prompt 6 — Car development UI and logic

```
Build the car development module in src/components/game/cardev/ and wire it to
src/stores/slices/carDevSlice.ts.

UI:
1. CarDevScreen.tsx — tabs: "Current Car" | "R&D" | "Parts Factory" | "Next Year's
Car".
2. CarOverview.tsx — visual chassis diagram (static SVG open-wheeler silhouette
with clickable hotspots for each PartSlot). Tapping a slot opens PartDetail.
3. PartDetail.tsx — shows fitted part stats (performance, condition/laps
remaining), inventory alternatives, "Fit this part" button.
4. RandDTree.tsx — MVP: linear per slot (4 tiers). For each slot, show 4 tier
cards. The current tier is unlocked; next tier shows cost, duration, projected
gain ±20% risk, "Start Research" button. Previously completed tiers dimmed.
5. PartsFactory.tsx — queue view. Shows in-progress parts with progress bar, weeks
remaining. "Cancel" button refunds 50%.
6. NextYearCar.tsx — slider allocating % of factory capacity to "this season" vs
"next season's chassis design". Tooltip explains the tradeoff.

Logic:
- startUpgrade(slot, tier) action: validates balance, debits cost, creates Upgrade
entity with weeksRemaining.
- advanceUpgradesWeekly: called by careerSlice.advanceDay when week boundary
crossed. Each active upgrade decrements weeksRemaining by (1 +
chiefDesignerSkill/100). On completion: rolls actual gain around projected ± risk,
```

```
creates a Part, marks upgrade complete.
- End-of-season carryover: each slot's best-condition Part becomes the baseline
for next season's starting car. Implemented in careerSlice.completeSeason.

Rules:
- All numbers and thresholds centralized in src/sim/carDevConstants.ts.
- Framer Motion for tab transitions only.
- Respect currency formatting (integer cents → $X,XXX,XXX display) via
src/lib/format.ts.

Test: player starts Tier 1 Engine upgrade at $2M over 4 weeks → advances 4 weeks →
Part created with performance in projected range.
```

Prompt 7 — Driver market and contract negotiation

```
Build src/components/game/market/ for the driver market + contract negotiation.

UI:
1. DriverMarketScreen.tsx — filterable table of all drivers (free agents and under
contract). Filters: Nationality, Age, Stat threshold (pace), Contract status.
Columns: Portrait, Name, Age, Nationality, Team (or "Free Agent"), Overall (avg of
5 stats), Contract expiry, Salary.

2. DriverProfile.tsx — modal showing full stats with bar charts (shadcn progress),
career record, current contract, potential ceiling (hidden unless scouted).

3. ContractNegotiation.tsx — modal with sliders:
   - Salary (driver's demand shown; ±20% acceptable band)
   - Length (1–4 seasons)
   - Podium bonus (0 / $100k / $500k / $1M per podium)
   - Signing bonus
   Each change updates a "Driver Willingness" meter (0–100%). Based on team
prestige, driver morale, market demand, competing offers (RNG'd), and salary
match. Below 50%: they'll reject. 50-80%: they'll counter-offer once. Above 80%:
accept.

4. ScoutingPanel.tsx — assign scouts to regions/tiers. Weekly, scouts reveal
hidden potential of drivers in their region. Scout skill affects reveal chance.

Logic:
- negotiateContract(driverId, terms): returns { status: 'accept' | 'counter' |
'reject', counterOffer?: Terms }.
- signDriver action (already in driverSlice): executes the cross-slice atomic
sign.
- Driver-initiated offers: once per month, a free agent or discontent driver may
send an unsolicited offer to the player's team. Shown as an interrupt.
```

```
- Silly season: at season end, all expiring contracts enter a 4-week negotiation
window with simulated AI team offers driving up salaries.

Rules:
- Counter-offers store the last rejected terms to avoid spam.
- Prestige gates top drivers: a driver with prestige demand 80 won't accept a team
below prestige 70.
- Show clear reasons on rejection: "Salary too low" / "Team not competitive
enough" / "Prefers a bigger team" / etc.

Test: attempt to sign a top driver with a bad team → reject → improve team
prestige → retry → accept.
```

Prompt 8 — Financial management and sponsor system

```
Build src/components/game/finance/ and extend src/stores/slices/financeSlice.ts.

UI:
1. FinanceDashboard.tsx — current balance (large), this-season income chart
(Recharts or a simple SVG bar chart), this-season expense chart by category, 4-
week forecast, ledger (recent transactions table with filters by category).

2. SponsorsScreen.tsx — 4 slot grid (title / premier ×2 / major×3). Each slot:
current sponsor name + logo placeholder + per-season value + remaining seasons.
Empty slots show "Browse Sponsors" button.

3. SponsorMarket.tsx — list of available sponsors filtered by your team's
marketability. Each card: Name, Tier (stars), Base payment, Bonus structure (per
podium / per win / per championship), Requirements (e.g. "Must finish top 6
Constructor"), Contract length. Tap → SponsorNegotiation.

4. SponsorNegotiation.tsx — single "Accept" / "Reject" for MVP. v1.1 adds
negotiation sliders.

Logic:
- Every race weekend: trigger sponsor-bonus payouts based on results (per-podium,
per-win, per-pole). Add to ledger.
- Every season end: title + premier + major sponsors pay their seasonPayment lump,
then contracts tick one season, expired contracts return sponsors to the market.
- Marketability is computed daily as: (avg of two drivers' marketability + team
prestige/2) / 1.5. Drives which sponsors are accessible.
- Board expectation check: at season end, if constructor position >
boardExpectation AND boardExpectation.patience > 0, decrement patience. At
patience 0, trigger "fired" interrupt → game over OR accept demotion.

Sponsor data seeding: write src/data/sponsors.json with 40 fictional sponsors
```

```
distributed across tiers. Use names in the style "Nordic Bank," "Vuccini,"
"Stonewall Technologies," "Iceni Security," "Condor Telecoms" — 2-word,
nationality-flavored, NOT real brands. Include the required Lanham-safe disclaimer
in a header comment.

Rules:
- All money arithmetic in integer cents.
- Every financial write goes through addTransaction() with a memo string.
- Display formatting via src/lib/format.ts exporting formatMoney(cents).

Test: start a race with 3 sponsors with podium bonuses → finish 1st + 3rd → verify
3 bonuses credited (2nd driver no podium, 1st gets podium bonus, 1st and 3rd each
earn per-podium from each bonus sponsor).
```

Prompt 9 — Season calendar and championship standings

```
Build src/components/game/calendar/ and wire to careerSlice.

UI:
1. CalendarScreen.tsx — vertical list of all races in the current season. Each
row: round number, circuit name + country flag, date, weather forecast icon, "Next
Race" chip on the upcoming one, result chip (driver finish position) on past
races. Tap a past race → RaceResultScreen. Tap next race → RaceWeekendScreen.

2. StandingsScreen.tsx — tabs: Drivers | Constructors. Shadcn Table: position,
name (or team), points, wins, podiums. Highlight player team/drivers. Use team-
color dot.

3. ContinueButton.tsx — persistent floating button bottom-right on most screens.
Triggers useGameStore.getState().advanceUntilInterrupt(). Shows the next interrupt
date in small text.

4. InterruptDialog.tsx — shadcn Dialog. Different content per interrupt type:
   - 'race' → navigate to RaceWeekendScreen
   - 'email'/'news' → show event with handle button
   - 'contractOffer' → navigate to contract modal
   - 'boardMeeting' → show warning/praise
   - 'upgradeComplete' → toast + link to cardev
   - 'transferDeadline' → warning

5. RaceResultScreen.tsx — post-race summary: top 10 finishers with team colors,
gap, fastest lap badge, pole badge, championship-impact delta, news headline,
optional lap-trace graph if stored.

Logic:
- advanceDay increments currentDate by 1. Processes scheduled events. Triggers
```

```
weekly jobs on Sunday→Monday boundary (sponsor payouts weekly, upgrade progress,
driver training, scouting).
- advanceUntilInterrupt loops advanceDay until nextInterrupt is non-null OR max 30
days safety cap.
- completeSeason: computes final standings, awards title bonuses, rolls driver
morale based on results vs expectations, triggers sponsor-expiry wave, runs AI
driver-market simulation, generates next season's race calendar (10 races with
randomized weather/temps from circuit volatility), creates next-year car with
carryover parts, rolls over any board patience changes. Pushes a "Season Ended"
interrupt for player recap.

Rules:
- Dates are ISO strings everywhere. Use date-fns for math.
- Championship standings recomputed after every race and stored in the
championship entity, so UI is O(1) read.
- All RNG for scheduled events uses splitmix32 derived from saveGame.rngSeed +
date string for determinism.

Test: start season → advanceUntilInterrupt → land on first race → complete →
advance → land on next race → after 10 races, advanceUntilInterrupt triggers
completeSeason and a new season is created.
```

Prompt 10 — CLAUDE.md for the project

```
Create CLAUDE.md at the project root with the full context a future Claude Code
session needs. Include these sections verbatim:

# Pit Lane Manager — Claude Code Context

## Project
Mobile F1 management sim (unlicensed) built on React 18 + Vite + TypeScript +
Tailwind v4 + shadcn-ui + Zustand + Framer Motion + Capacitor 7. Forked from
Dynasty Manager (football management game). Target: iOS App Store + Google Play,
free-to-play with IAP.

## Legal boundary
Never introduce "F1", "Formula 1", "Formula One", "FIA", "Grand Prix" as brand
elements. Real country/city names are OK. Real circuit names are NOT (they're
trademarked — use our fictional circuit names from data/circuits.json). Real
driver names are NEVER OK. Never match real team three-letter codes (FER, MER,
MCL, RBR, ALP, AST, WIL, ALF, HAA, SAU). All driver portraits/helmets/suits must
be original art.

## Architecture invariants
- Single bound Zustand store (useGameStore) with slices combined under persist +
immer + devtools. NEVER create additional independent Zustand stores.
```

- All entity IDs are branded strings. Never pass raw string literals as IDs.
- All money is integer cents (branded Money type). Convert to display at render time only via lib/format.ts formatMoney().
- All stats are branded 0-100. Use makeStat() at boundaries.
- All entities normalized as Record<Id, Entity>. No nested entity references.
- The race sim (src/sim/) is pure-functional — no React, no Zustand, no side effects. RNG is always passed in.
- Hydration gate is mandatory: App.tsx must not render game content until useHydration() === true, or iOS will reject with 2.1 crash on launch.
- Framer Motion: only animate transform/opacity. Never width/height/top/left. Wrap race screen in LazyMotion.
- During race tick loop, NEVER bind Zustand state directly to Framer Motion animate prop. Push to UI only at lap boundaries.

Folder structure
(Paste the full folder tree from the migration plan)

Commands
- `npm run dev` — Vite dev server
- `npm run build` — production build
- `npm run test` — Vitest
- `npx cap sync` — after any native config change
- `npx cap run ios` / `android` — run on device/emulator
- `npm run lint` — ESLint

Save migration rules
Every schema change to SaveGame requires:
1. Bump SaveGame.version in src/types/game.ts
2. Add a migration case in useGameStore persist config migrate(). Must be idempotent.
3. Update any v1-saves test fixture in src/stores/__tests__/migrations.test.ts.

IAP rules (RevenueCat)
Never gate game progression behind currency. No loot boxes. No energy system. All IAPs fall in one of: Remove Ads (non-consumable), Credit Bundles (consumables), Season Pass (non-renewing subscription), Livery/Cosmetic Packs (non-consumables), Manager Pro (auto-renewing subscription).

Commit conventions
Conventional Commits. Prefixes: feat:, fix:, chore:, refactor:, docs:, test:.

Performance budgets
- Cold launch to hydrated home screen: <3s on iPhone SE 2 / mid-range Android.
- Race tick at 1x: 60fps sustained, no dropped frames on iPhone SE 2.
- Save write: <200ms for a 1MB save.

```
Write this file now, then git commit with message "docs: add CLAUDE.md project
context".
```

Prompt 11 — TASK.md session continuity template

```
Create TASK.md at the project root as a live working document. Structure:

# Current Task

## Goal
(One-line statement of what I'm working on this session)

## Context
(Links to relevant files, previous commits, open PRs)

## Plan
- [ ] Step 1
- [ ] Step 2

## Blocked on
(Anything external or decision-needed)

## Session log
- YYYY-MM-DD HH:MM — started
- …

## Definition of done
- [ ] Tests green
- [ ] `npm run build` green
- [ ] `npx cap sync ios && npx cap sync android` green
- [ ] Manual test on simulator passed
- [ ] Committed with Conventional Commit message
- [ ] TASK.md reset for next session

# Backlog
(Ordered list of next 10 tasks)

# Done (last 20)
(Archive)

Seed the Backlog with the week-by-week tasks from the 8-week roadmap. Seed Done
with "Forked from Dynasty Manager" as the first entry.
```

Week 1 — Foundation

Fork Dynasty Manager, apply Prompts 1–3. Delete football modules, rename entities, scaffold TypeScript types, stand up the Zustand store with Capacitor Preferences persistence. Validate that the app boots on iOS simulator + Android emulator and persists/restores a dummy save. **Exit criteria**: hydration gate working, empty-state home screen renders, save round-trips.

Week 2 — Race sim engine

Apply Prompt 4. Build the pure-functional sim in `src/sim/` with mulberry32 RNG, lap-time formula, 4-phase tire model, overtake sigmoid, pit decision tree, Markov weather, reliability rolls, and a `simulateRace` entrypoint. Write Vitest determinism + sanity tests (1000 iterations: better car wins ⩾60%, same seed = identical result). **Exit criteria**: can call `simulateRace()` from a console and get a plausible 10-lap finishing order in <50ms.

Week 3 — Live race UI

Apply Prompt 5. Build RaceScreen, TrackMap (simplified per-circuit SVG), StandingsTable, PlayerHUD, StrategyPanel, TimeControls, RaceEvents, useRaceLoop hook. Wire to raceSlice actions. Haptic feedback on strategy actions. **Exit criteria**: can run a full 10-lap race at 1x/2x/5x with live pit calls and tire changes, 60fps on iPhone SE 2.

Week 4 — Car development + driver market

Apply Prompts 6 and 7. Build cardev tree (linear 4-tier per slot), parts factory, next-year's-car slider; build driver market, contract negotiation modal, scouting panel. Seed drivers.json + circuits.json with the 20 drivers and 10 circuits from Section 2. **Exit criteria**: can start a Tier 1 Engine upgrade, advance 4 weeks, receive the completed Part, fit it, see performance gain in the next race; can sign a free-agent driver.

Week 5 — Finance, sponsors, calendar, standings

Apply Prompts 8 and 9. Build FinanceDashboard, SponsorsScreen, SponsorMarket, CalendarScreen, StandingsScreen, ContinueButton, InterruptDialog, RaceResultScreen. Seed sponsors.json with 40 fictional sponsors. Wire advanceUntilInterrupt + season lifecycle. **Exit criteria**: can play a full 10-race season end-to-end with sponsor payouts, championship standings, season-end summary, and a rolled-over next season.

Week 6 — Onboarding, polish, IAP integration

Build new-game flow (team select → custom team name + color picker → starting budget based on chosen team tier), tutorial overlays on first race weekend, news feed, settings screen (audio, haptics, language toggle), about/credits screen with legal disclaimer. Integrate **@revenuecat/purchases-capacitor**, configure products in App Store Connect + Play Console (Remove Ads, Starter Pack, credit bundles ×7, Season Pass, Founder's Pack), implement paywall UI, add Restore Purchases button. Add Privacy Manifest `PrivacyInfo.xcprivacy` declaring

UserDefaults (CA92.1), FileTimestamp, DiskSpace, SystemBootTime; `NSPrivacyTracking=false`. **Exit criteria**: a sandbox IAP completes end-to-end on both platforms; app passes internal dogfooding on 3 devices.

Week 7 — Soft launch

Ship v0.9 to **Philippines + Indonesia** (Google Play internal → closed testing → production in those two regions only; App Store phased rollout in PH + ID). Integrate GameAnalytics (free). Create Discord server. Begin creator outreach (Tommo F1, Bradder, TheTias, Between the Lines F1 — send builds + promo codes 2–3 weeks before global). Monitor D1/D7 retention, ARPDAU, funnel, crash rate. **Targets**: D1 ⩾35%, D7 ⩾15%, crash-free ⩾98%. Fix top 3 player complaints from reviews.

Week 8 — Global launch prep + submit

Expand soft launch to Netherlands + Canada for monetization signal. Tune economy based on PH/ID data (adjust starting budget, upgrade costs, sponsor values). Finalize store listings: 1290×2796 (iPhone 6.9") and 2064×2752 (iPad 13") screenshots (5 each with the mandated captions "Win With Pure Strategy" / "Call the Perfect Pit Stop" / "Your Team, Your Colors" / "No Loot Boxes. Ever." / "Build Your Racing Dynasty"), 1024×500 feature graphic, 512×512 adaptive icon, 60-second trailer. Title "Pit Lane: Racing Manager" / subtitle "Grand Prix Team Strategy" / keyword field `formula,motorsport,racing,tycoon,pitstop,tyre,championship,driver,endurance,gt,openwheel,simulator`. Submit to App Store (expect 48–72h review) and Google Play (expect 7 days for new personal account + 14-day closed testing gate). Schedule Apple In-App Event for launch day to boost featuring odds. **Global launch Day 0**: Reddit posts (r/MotorsportManager, r/iosgaming, r/AndroidGaming, r/formula1 if mod-approved), Product Hunt launch, Discord Founder role drops, $500/day Apple Search Ads on "motorsport manager" + "racing manager" + "f1 game," press outreach to 148Apps / Pocket Tactics / Pocket Gamer / TouchArcade.

MVP cut list (explicitly save for v1.1+)

Tiers 2 + 3 and promotion/relegation • sprint weekends • reserve drivers • branching R&D tree • mechanics and strategist staff sub-roles • 9-stat driver system (start with 5) • livery pattern editor (ship with 2-color picker only) • create-a-driver • historical seasons • async co-op leagues • TikTok-ready replay share • additional circuits beyond 10 • additional languages beyond English at first submission.

Testing strategy for Capacitor

Dev loop on `npm run dev` in desktop Chrome with mobile emulation for the 90% of UI work. Validate on-device at each milestone: iPhone SE 2 (worst-case iOS WebView), iPad 11", Pixel 6a (mid-range Android), a Samsung budget device (Android WebView edge cases). Use Safari Web Inspector + Chrome remote DevTools for profiling. Vitest for pure sim logic — aim for 80% coverage on `src/sim/`. No E2E tests for MVP; rely on on-device manual test scripts stored in `docs/testing.md`.

Submission checklist

App Store: App Store Connect app created, Bundle ID matches capacitor.config.ts, all IAP products "Ready to Submit," Restore Purchases button visible, Privacy Manifest present, Privacy Policy URL live, age rating questionnaire answered (all None → 4+), screenshots for 6.9" + 13" iPad, App Preview video ≤30s, reviewer demo credentials in review notes, Sign in with Apple NOT offered (we have no social login — avoid the requirement). **Google Play**: developer account closed-testing-with-20-testers prerequisite complete, AAB targets API 35, Play Billing Library 7.0.0+, Data Safety form = "No data collected" (assuming no analytics; if GameAnalytics added, declare device IDs + diagnostics), Content Rating = IARC 3+, Privacy Policy URL, feature graphic 1024×500, phone screenshots 2–8, $25 registration fee paid, Play App Signing enrolled.

---

7. Competitive positioning

The differentiation thesis

Three fatal gaps exist in the current mobile racing-management market: **MM4 is frozen in maintenance, F1 Clash is pay-to-win with loot boxes, and F1 Manager orphaned console-sim fans when Frontier cancelled it**. Pit Lane Manager targets all three orphan audiences with one product. Against MM4: ship the features MM4 fans have begged for since 2023 (custom race lengths, interval timing, opponent stats visibility, weather graph, stronger AI, fixed leaderboard names) and match its depth-without-overwhelm feel. Against F1 Clash: a proper single-player career with no gacha and no pay-to-win, exploiting the ASA 2023 loot-box ruling as marketing ammunition. Against F1 Manager: a livery editor + create-a-driver + historical seasons — the modes Frontier's audience loved but Frontier cancelled.

Pricing vs competitors

F2P with entry IAP at **$2.99 Starter Pack** (vs F1 Clash's $4.99+) and **$5.99 Season Pass** (vs F1 Clash's $9.99 Premium Pit Pass). Manager Pro subscription at **$4.99/mo or $34.99/yr** sits below iGP Manager's VIP tier and positions as "fair" rather than essential. No IAP is required to win races or climb tiers — the economy balances so an ad-free player catches up within 2 weeks. Publish the economy rules in a blog post for transparency.

ASO strategy

Title/Subtitle variants to A/B test:

- **A (search-optimized):** `Pit Lane: Racing Manager` / `Grand Prix Team Strategy`
- **B (brand):** `Apex Team: F1 Manager Sim` / `Pit Stop & Tire Strategy`
- **C (aspirational):** `Pole Position Manager` / `Build Your Racing Dynasty`

Apple keyword field (100 chars):

```
formula,motorsport,racing,tycoon,pitstop,tyre,championship,driver,endurance,gt,ope
```

`nwheel,simulator`). **Do NOT use "F1," "Formula 1," or "Grand Prix"** in the title or subtitle — trademark risk is real and Apple has pulled apps over similar claims. The subtitle "Grand Prix" as a descriptive phrase in lowercase in the full description is generally safe. Screenshots' first three are the install decision: **"Win With Pure Strategy"** (race action), **"Call the Perfect Pit Stop"** (strategy panel), **"No Loot Boxes. Ever."** (marketing differentiator — a direct attack on F1 Clash). Localize at launch in English, Brazilian Portuguese, Spanish, German, Italian, French, Japanese — MM4 shipped only 6 languages and under-serves LATAM/Japan; this is cheap upside.

Launch and community

Soft launch Jan 20, 2026 in PH + ID → Feb 3 expand to NL + CA → Feb 24 global, timed for the March Australian GP opener. Use the race-weekend DAU spike to capture organic search. Creator outreach targets Tommo F1, Bradder, TheTias, Between the Lines F1, and aarava (tier up if any agree); offer exclusive early builds 2 weeks pre-launch with Founder codes to give to their audiences. Discord server must launch **before** the soft launch with #race-weekend-chat tied to real F1 calendar, #livery-showcase (player creations go viral on Twitter/TikTok), and #strategy-tips. Reddit rollout: r/MotorsportManager (tight-knit ~8K, highest conversion), r/F1Manager (orphaned Frontier audience, high conversion), r/iosgaming + r/AndroidGaming (weekly self-promo threads), r/formula1 only via mod-approved or promoted posts. TikTok is the dark-horse channel — short "what-if strategy moment" clips during real race weekends, using your UI screenshots with race commentary overlay, can go six-figure viral for near-zero CAC.

Bottom line

The plan closes the loop: a legally-cleared unlicensed F1 sim, forked from a proven React/TS/Capacitor foundation, shipped free-to-play with non-predatory IAP into a 826M-fan market that has exactly one frozen incumbent and one gacha-heavy licensed product. Eight weeks is aggressive but feasible because **70% of Dynasty Manager's systems map directly to F1 management** and the 30% that don't — the race engine and car development — are bounded, well-specified problems with working reference algorithms and open-source precedents. The most leveraged work is **week 2 (the sim) and week 6 (IAP + privacy manifest)**; slippage there kills the window. The biggest existential risk is not competition — it's an App Store rejection under Guideline 2.1 (crash on launch) caused by a missing hydration gate or 3.1.1 (IAP misconfigured); both are checkpoints in the prompt library and the submission checklist. Ship by late February 2026 for the Australian GP tailwind, commit publicly to "no loot boxes, no pay-to-win," and let the F1 Clash refugees come to you.